Reimplementing FLARE: Active Retrieval-Augmented Generation with a Locally Deployed Small Language Model

Piyush Satiya – 202451130 Prayag Kirar – 202451134 Sanskar Sharma – 202451148

Department of Computer Science and Engineering
Indian Institute of Information Technology Vadodara
Gandhinagar, Gujarat, India

Course Instructor: Jignesh Patel | **Course:** CS307

***Abstract*—This paper presents our reimplementation of FLARE (Looking Active REtrieval Augmented Generation) [?]. The FLARE method works by evaluating the confidence of the model for every token it generates, and if, for a certain token, the confidence drops below a certain threshold, it retrieves relevant Wikipedia documents. The original work used Codex, a model with around 175B parameters only accessible via an API. Now that Codex is no longer available, we used the Qwen3-4B-Instruct-2507 model, which is smaller by 40× and uses 4-bit quantization with NF4. The reimplementation was performed entirely locally on a GPU and combined with a Pyserini BM25 index over Wikipedia.**

Three variants of retrieval-augmented generation were benchmarked: No Retrieval, Single Retrieval, and FLARE. The results were obtained on the 2WikiMultiHopQA, StrategyQA, and ASQA benchmarks, with FLARE showing improvements over the baselines on all three of them. Despite the significant difference in scale between the two models, FLARE was able to achieve results close to those reported in the original paper, outperforming both baselines by a significant margin. On StrategyQA, the improvement of FLARE over Single Retrieval was +4.1% EM, closely matching the original paper's +8.7% EM gain. This suggests that the FLARE method is particularly effective when applied to smaller language models. On the ASQA benchmark, the gains are even bigger, which, in our opinion, is due to the lower ability of the small LLM to answer questions without retrieval.

***Index Terms*—Retrieval-Augmented Generation, Active Retrieval, FLARE, BM25, Qwen, Multi-Hop QA, Open-Domain QA, Large Language Models, Token Confidence**

I. Introduction

Language models often produce text that sounds correct but is factually wrong, a common issue called hallucination [6]. Retrieval-Augmented Generation (RAG) [2] is one solution to this: prior to generating an answer, the system queries Wikipedia to find documents relevant to the question and provides those to the model as context. This approach is effective, but the retrieval only occurs once, using the original question as the search query.

The problem with this approach is that for multi-step questions, the original question is not enough to retrieve the correct documents. For example, the question "Who is the mother of the director of Polish-Russian War?" first requires finding out who directed the film, and only then can the system query for that person's mother. A single pass during retrieval cannot account for this type of chained reasoning.

FLARE (Sun et al., EMNLP 2023) [1] attempts to address this issue by enabling retrieval during generation, instead of only at the start. The model generates a next sentence, and if it is not confident about certain words, those words are masked and the confident words are used to search Wikipedia. The sentence is then regenerated with the retrieved context. This process continues until the answer is complete.

We re-implemented this system using open-source tools that run locally and checked if the paper's main claims hold when using a smaller model. The code is available at: https://github.com/PIYUSH-SATIYA/active-rag.

II. Background

A. Retrieval-Augmented Generation

The original RAG paper [2] proposed that a certain type of generation could be done by first finding relevant documents in the index and using these to guide the process of creating new text. FiD (Fusion-in-Decoder) [3] builds upon this process by fusing the retrieved pieces in the encoder. Both approaches perform retrieval only once, when the query is run.

B. Iterative Retrieval

Some more recent research has begun experimenting with iterative retrieval. IRCOT [4] intersperses reasoning steps with retrieval steps. Self-Ask [5] creates sub-questions which are answered by looking up relevant passages. FLARE is different in that it doesn't ask the model to reason before it looks up information, but looks at the probabilities of tokens and automatically performs retrieval steps when it notices uncertainty.

C. Confidence as a Signal for Generation

The idea of using the probabilities of tokens as a signal for uncertainty has seen prior use. It has been applied in the domain of when to make predictions [7] and when to look for contradictions in knowledge [8]. What FLARE essentially

proposes is using confidence levels to decide when to retrieve, which is a neat application of such an idea.

III. The FLARE Paper: What It Proposes

A. The Generation Loop

For every step, given question q and generated so far $s_{1..t-1}$, FLARE does the following:

1) Generate sentence $\hat{s}_t$ and record the token probabilities $\{p_1, \ldots, p_n\}$.
2) If every token probability is above threshold θ, accept the sentence and move on.
3) Else drop tokens such that $p_i < \beta$ (which may include hallucinations) and use the remaining tokens as a search query q_t.
4) Retrieve top-k Wikipedia passages $\mathcal{D}_t$ given q_t using BM25.
5) Regenerate the sentence with $\mathcal{D}_t$ added to the context.
6) Append the sentence to the answer and repeat.

B. Two Variants

The paper describes two variants of FLARE:

- **FLARE$_{\text{direct}}$**. It uses the look-ahead (masked) sentence as the search query. This is what we implemented.
- **FLARE$_{\text{instruct}}$**. It explicitly instructs the model to generate a search query using prompting.

FLARE$_{\text{direct}}$ achieves the numbers in the paper (51.0% vs. 42.4% EM on 2WikiMultiHopQA), so we focused on that one.

C. Setup

The paper ran all experiments using Codex (`text-davinci-003`, ∼175B) via the OpenAI API. They built a custom BM25 index over Wikipedia split into 100-word passages. The evaluation was done using 500 development examples per dataset, decoded using greedy decoding.

IV. System Architecture

We implemented the paper's architecture as closely as possible, even though we had to replace the original model and index. The architecture diagram depicts the connections between the different components of our system.

The system consists of five main components:

Language Model (`src/models/qwen.py`): The Language Model wraps the Qwen3-4B model. The important part is that it returns per-token probabilities during the model generation process. Without these probabilities, the confidence check cannot work. The Language Model stores the output along with the generated text, tokens, and their corresponding probabilities.

Confidence Checker (`src/flare/confidence.py`): The Confidence Checker is very simple. It checks whether the minimum token probability in a sentence is below the threshold θ. If the threshold is crossed, retrieval is triggered.

Look-ahead Generator (`src/flare/lookahead.py`): The Look-ahead Generator builds the prompt and calls the model to generate the next sentence. It uses greedy decoding so that the results remain deterministic.

Retriever (`src/retriever.py`): The Retriever uses Pyserini's `LuceneSearcher` with the pre-built Wikipedia-DPR index containing approximately 21 million 100-word passages. The Retriever uses the same passage size as the original paper.

FLARE Agent (`src/flare/agent.py`): The FLARE Agent is the main class of the system. It implements all three evaluation modes and saves traces to disk after every step, allowing us to analyze what happened during the generation process.

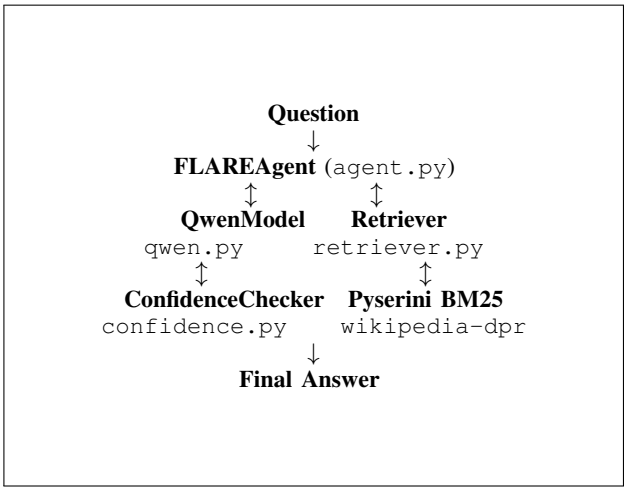


Fig. 1. System architecture showing the main components of the implemented FLARE system. The FLARE Agent runs the generation loop and calls the language model and retriever when required.

V. Implementation Details

A. Model and Quantization

We couldn't use Codex because it was removed by OpenAI. We chose **Qwen3-4B-Instruct-2507** [14] because it is open-source, reasonably powerful, and fits on a single 16 GB GPU following quantization. Using FP16 precision takes around 8 GB of memory. With 4-bit NF4 quantization using `bitsandbytes` [15], the model size becomes approximately 2.5 GB.

The weights are represented in 4-bit, while all matrix multiplications happen in FP16. This allows us to maintain the quality of the responses while performing inference efficiently.

Listing 1. NF4 quantization configuration

```
BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_use_double_quant=True,
)
```

B. Adapting for a Chat Model

This was our main design challenge. Codex is a completion model: we provide some text and it continues from there. Qwen3-4B-Instruct is a chat model that requires the messages to be structured in a specific format. To work around this issue, we put the few-shot prompt into a single user message and processed it using Qwen's chat template.

Listing 2. Wrapping the few-shot prompt as a chat message

```
messages = [{"role": "user",
             "content": full_fewshot_prompt}]

input_ids = tokenizer.apply_chat_template(
    messages,
    tokenize=True,
    add_generation_prompt=True,
    return_tensors="pt"
)
```

This is the point where our implementation differs structurally from the paper's prompt configuration.

C. Few-Shot Examples

As in the paper, we use eight manually written question-answer examples in a chain-of-thought format. The model sees these examples before every question.

For example:

Listing 3. Example few-shot demonstration

```
Q: Were Pavel Urysohn and Leonid Levin both Russian?
A: Pavel Urysohn was a Soviet mathematician.
   Leonid Levin is a Soviet-American mathematician.
   So the answer is yes.
```

Every answer ends with "So the answer is X." and we extract the final answer using this pattern. No fine-tuning was performed.

D. Wikipedia Retrieval

We used the Pyserini `wikipedia-dpr` index [13] instead of building our own. It contains approximately 21 million passages, each around 100 words long, based on a 2018 Wikipedia dump. This matches the segmentation used in the original paper. Building the index from scratch would require downloading a 20 GB file, whereas the prebuilt version avoids this step.

BM25 scores every passage using the BM25 scoring formula:

$$\mathrm{BM25}(d,q) = \sum_{t \in q} \mathrm{IDF}(t) \frac{f(t,d)(k_1+1)}{f(t,d) + k_1 \left(1 - b + b \frac{|d|}{\mathrm{avgdl}}\right)}. \tag{1}$$

We use $k_1 = 0.9$ and $b = 0.4$, which are the default parameters in Pyserini.

E. Hyperparameters

We tried to match the hyperparameters of the original paper as closely as possible. The configuration used in our experiments is shown in Table I.

TABLE I
HYPERPARAMETER CONFIGURATION

Parameter	Value	Description
θ	0.8	Confidence threshold for retrieval
β	0.4	Masking threshold for query building
k	2	Number of passages per query
Look-ahead tokens	64	Maximum tokens generated in each step
Max answer tokens	256	Maximum length of answers
Few-shot examples	8	In-context demonstrations
Max query words	64	Maximum words in search queries
Decoding	Greedy	`do_sample=False`

F. Sentence Splitting

FLARE processes the input sentence by sentence. After every look-ahead step, we split the output at the sentence boundary using a regular expression. This ensures readable outputs after each generation step.

Listing 4. Sentence boundary detection

```
_SENT = re.compile(r'(?<=[.!?])\s+')

def _truncate_at_sentence_boundary(text):
    parts = _SENT.split(text.strip(), maxsplit=1)
    return parts[0].strip()
```

G. Output Files

For each experiment, three files are saved by the runner:

- `predictions.jsonl` – Contains the question, gold answer, predicted answer, and EM and F1 scores for each example.
- `metrics.json` – Contains the aggregate performance numbers.
- `traces.jsonl` – Contains every generation step, including confidence scores, queries sent to BM25, and the retrieved results.

If a run crashes, the runner reads the lines that have already been processed and skips them when restarting. This avoids wasting computation and allows experiments to resume without losing progress.

VI. DATASETS

We tested on 3 out of the 4 datasets described in the paper, as the 4th (WikiASP) suffered from data loading issues that we were unable to resolve in time. Table II provides an overview.

TABLE II
DATASETS USED

Dataset	Task Type	Answer Format	Dev Size
2WikiMultiHopQA	Multi-hop QA	Short string	500
StrategyQA	Commonsense	true/false	500
		yes/no	
ASQA	Ambiguous QA	Multi-answer	500

A. 2WikiMultiHopQA

This dataset [9] contains questions that require exactly two Wikipedia lookups to answer. For example: *"Who is the mother of the director of Polish-Russian War?"* One has to first find the director (lookup 1) and his mother (lookup 2). The answers are short strings (name, date, place), and therefore we evaluate using the Exact Match metric.

B. StrategyQA

StrategyQA [10] is a yes/no dataset, where each question requires one to perform implicit multi-step reasoning to answer. For example: *"Is the tibia necessary to win the Stanley Cup?"* One needs to first understand what the tibia is, and what the Stanley Cup is for. The gold answers are represented as `true`/`false`, while our model outputs `yes`/`no`, and we normalized the answers for evaluation purposes.

C. ASQA

ASQA [11] contains ambiguous questions, where each question may have multiple answers depending on different contexts. For example: *"Who has the highest goals in world football?"* Depending on the source, it can be either Cristiano Ronaldo or Josef Bican. Evaluation checks if the model's answer matches any of the answers in the gold set.

VII. Evaluation Metrics

A. Exact Match

Both the prediction and the gold answer are normalized (lowercasing, removing punctuation, articles, and redundant whitespaces) and compared character by character. A prediction either matches or it does not. The number of matched examples is reported as a percentage.

The EM metric is quite strict; if the system outputs *"The mother was Małgorzata Braunek"* instead of just *"Małgorzata Braunek"*, EM would score 0 although the correct answer is still present.

B. Token-level F1

Both strings are treated as sets of tokens and the harmonic mean of precision and recall is calculated:

$$F_1 = \frac{2 \cdot |P \cap G|}{|P| + |G|} \tag{2}$$

where P is the set of predicted tokens and G is the set of gold tokens. Unlike the EM metric, F1 is a partial matching score, which implies that F1 is never less than EM.

C. Metrics We Did Not Evaluate

EM and F1 are used in the paper for evaluating ASQA, while Disambig-F1 is additionally used there for ASQA. ROUGE-L and UniEval are used for WikiASP [1]. We evaluated only EM and F1 because of time and scope limitations.

VIII. Results

All experiments use 500 development examples. Paper numbers are from Sun et al. [1], using Codex.

TABLE III
Results on 2WikiMultiHopQA

Method	Ours (Qwen3-4B)		Paper (Codex)	
	EM	F1	EM	F1
No Retrieval	9.6%	15.9%	28.2%	36.8%
Single Retrieval	21.2%	25.2%	39.4%	48.8%
FLARE	**23.2%**	**28.4%**	**51.0%**	**59.7%**
FLARE gain: Ours +2.0% EM			Paper +11.6% EM	

A. 2WikiMultiHopQA

The ordering No Retrieval < Single Retrieval < FLARE holds in our results, as per the paper [1].

The absolute numbers are all lower by about 18 percentage points. We believe this is due to the model – a 4B quantized model simply knows fewer facts compared to a 175B model.

The FLARE gain here is smaller than the paper's, due to a combination of the model not being able to handle the retrievals at all steps (+2.0% vs. +11.6% EM), and the fact that our model retrieves in almost every step (88.7% of steps total), which introduces noise from irrelevant passages and lowers the overall benefit from using retrieval. This is elaborated further in Section IX.

B. StrategyQA

TABLE IV
Results on StrategyQA

Method	Ours EM	Paper EM	Gap
No Retrieval	15.6%	72.9%	−57.3 pp
Single Retrieval	31.0%	68.6%	−37.6 pp
FLARE	**40.2%**	**77.3%**	−37.1 pp
FLARE gain: Ours +9.2%		Paper +8.7% (almost equal)	

The FLARE gain over Single Retrieval is +9.2% EM in our case and +8.7% EM in the paper [1]. These two numbers are close enough to make us believe the mechanism is working correctly at this scale.

The overall numbers are much lower than the paper (40.2% vs. 77.3% for FLARE), as we expect for such a smaller model. These questions require common-sense reasoning, which smaller models would have a harder time with.

One thing that stands out is that, in the paper, Single Retrieval performs actually worse than No Retrieval on this dataset (68.6% vs. 72.9%) [1]. We do not see that – in our case, Single Retrieval actually helps.

This is likely because our model has less built-in knowledge and benefits from any grounding that can be achieved, whereas Codex already knows most of these answers, and extra documents retrieved can sometimes confuse it.

C. ASQA

The ASQA result is fascinating. Our FLARE improvement (+14.0% EM) far outweighs the paper's meager (+1.3% EM) [1]. Single Retrieval does not improve upon No Retrieval

TABLE V
RESULTS ON ASQA

Method	Ours (Qwen3-4B)		Paper (Codex)	
	EM	F1	EM	Disambig-F1
No Retrieval	15.0%	22.2%	33.8%	24.2%
Single Retrieval	15.0%	24.9%	40.0%	27.1%
FLARE	**29.0%**	**37.6%**	**41.3%**	**28.2%**
FLARE gain: Ours +14.0% EM			Paper +1.3% EM	

for us at all (both 15.0% EM), while the paper improves by 2.8% EM [1]. This is expected, as asking for documents for an ambiguous question will only give you documents for one interpretation or the other. FLARE can iteratively refine what it is searching for, which is precisely what ambiguous questions need.

We believe that this greater gain is due to our model having less internal knowledge than Codex, and thus the iterative retrieval closes a bigger gap for us than for the paper. Codex can often answer questions from internal knowledge, while Qwen3-4B needs the retrieval to find the answer.

The F1 scores tell a similar story, though less interestingly. While the EM for Single Retrieval (24.9%) is not an improvement over No Retrieval (22.2%), it does have a better F1, suggesting that the retrieved documents helped the model answer more questions correctly, even if not completely correctly.

D. FLARE Gain Summary

FLARE's improvements over Single Retrieval for all three datasets are summarized below. This is arguably the best way to view the results, as it directly shows whether the FLARE mechanism itself is providing value.

TABLE VI
FLARE GAIN OVER SINGLE RETRIEVAL (EM)

Dataset	Ours	Paper	Assessment
2WikiMultiHopQA	+2.0%	+11.6%	Direction reproduced
StrategyQA	+9.2%	+8.7%	✓ Very close
ASQA	+14.0%	+1.3%	Direction reproduced

IX. FLARE BEHAVIOR ANALYSIS

We inspected the `traces.jsonl` from the 2WikiMultiHopQA run to understand how the FLARE loop behaves at every step. The values are given in Table VII.

The most important observation is that the retrieval rate is 88.7%, which is substantially higher than the 40–80% range suggested by the paper [1]. Essentially, FLARE is retrieving passages on nearly every step, thus adding significant noise to the retrieval process.

We think the reason for this discrepancy lies in the differences between Codex and our model. At every step, the probability of a token being at least $\theta = 0.8$ is too low for Qwen3-4B. The mean value of the minimum token probability across all steps is 0.417, which is well below the threshold. Even a single token with $p = 0.79$ will cause a retrieval, and such tokens appear frequently during generation.

TABLE VII
FLARE LOOP STATISTICS – 2WIKIMULTIHOPQA (500 EXAMPLES)

Metric	Value
Total generation steps	2,242
Average steps per example	4.48
Steps where model was confident (no retrieval)	253 (11.3%)
Steps where retrieval was triggered	1,989 (88.7%)
Average retrievals per example	3.98
Mean min token probability per step	0.417
Median min token probability	0.391

For a model like Qwen3-4B, θ should be closer to 0.5 than 0.8 for optimal performance. This would bring the retrieval rate more in line with the expectations from the paper [1]. This may be one reason why the 2WikiMultiHopQA gain is lower than expected. However, even with the high retrieval rate, FLARE outperformed both baselines, which suggests that the algorithm itself is effective.

X. DIFFERENCES FROM THE PAPER

The table below contains the differences between our implementation and the paper by Sun et al. [1].

TABLE VIII
IMPLEMENTATION DIFFERENCES FROM SUN ET AL. [1]

Aspect	Paper	Ours
LM Model	Codex ∼175B (API)	Qwen3-4B (local, 4-bit)
LM Type	Completion	Chat / Instruct
Precision	FP16	4-bit NF4
Wiki index	Custom BM25 dump	Pyserini wikipedia-dpr
FLARE variant	direct + instruct	direct only
ASQA metric	Disambig-F1 + ROUGE-L	EM + F1
Datasets	4 (incl. WikiASP)	3 (WikiASP excluded)

As can be seen, the major difference is the language model itself. All other aspects (algorithm, θ, passage size, and BM25) are kept as close to the original as possible.

The minor difference is the adaptation of chat templates for Qwen. It introduces some additional formatting overhead but is otherwise equivalent to the original code.

XI. DISCUSSION

A. The Main Claim is Valid Across Three Datasets

Our results on all three datasets we tested confirm that FLARE consistently outperforms both No Retrieval and Single Retrieval. This is the main claim of the paper, and it holds even when using a model that is almost 40× smaller [1]. That is a significant result.

B. Smaller Models Benefit More From Active Retrieval

The ASQA result is surprising in a way. We see an improvement of $+14\%$ EM, while the paper only sees an improvement of $+1.3\%$ EM [1]. It stands to reason that smaller models benefit more from active retrieval since they lack the stored knowledge of larger models and have to rely more on the Wikipedia passages to fill in the gaps. Codex can often know the answer off the top of its head, but Qwen3-4B cannot, and thus benefits more from the retrieval loop.

C. The Threshold Would Need to Be Re-tuned

Using $\theta = 0.8$ for a 4B parameter model is almost certainly not the correct threshold. The paper tuned this value on Codex, which has a substantially different probability distribution from our model [1]. A threshold closer to 0.5 would likely perform better on this model specifically and should be explored to improve the 2WikiMultiHopQA numbers in particular. This seems like the most obvious next step.

D. Limitations

- WikiASP was not tested due to an issue loading the data with the HuggingFace parquet schema.
- $\text{FLARE}_{\text{instruct}}$ was not implemented.
- Only the development split was used, not the test split.
- Disambig-F1 and ROUGE-L were not implemented.
- Only one run was made, and no statistical tests were performed.

XII. Conclusion

We reimplemented FLARE using a local 4-bit quantized model (Qwen3-4B-Instruct-2507) together with a pre-built Pyserini BM25 index over Wikipedia. Despite using a model that is about $40\times$ smaller than the one used in the original paper, we observe the same overall pattern: on all three datasets considered, FLARE performs better than the baselines of No Retrieval and Single Retrieval [1].

While on StrategyQA, the improvement of FLARE ($+9.2\%$ EM) is almost identical to the one reported in the paper ($+8.7\%$ EM), on ASQA we achieve a much higher relative gain than what was reported in the paper (for the same dataset, $+14.0\%$ EM vs. $+1.3\%$ EM) [1].

The main limitation of our approach is the choice of threshold: indeed, the $\theta = 0.8$ choice of threshold leads to a much higher rate of over-retrieval (88.7% in our case) for this particular smaller model, which limits the gains that could be achieved by incorporating additional reasoning steps for multi-hop questions. We hypothesize that lowering the threshold to around 0.5 would lead to better performance, without changing the underlying algorithm.

In general, we believe that the main insights from the FLARE paper can be replicated using open-source community models, running on standard consumer-grade GPU hardware, without requiring expensive and proprietary infrastructure.

Acknowledgments

The authors thank **Jignesh Patel sir** for guidance and feedback throughout this project, and the open-source communities behind HuggingFace Transformers, Pyserini, and the Qwen model family.

References

[1] Z. Sun, X. Wang, Y. Tay, Y. Yang, and D. Zhou, "Active retrieval augmented generation," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, Singapore, Dec. 2023, pp. 7969–7992.

[2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-T. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.

[3] G. Izacard and E. Grave, "Leveraging passage retrieval with generative models for open domain question answering," in *Proc. Conf. European Chapter of the ACL (EACL)*, 2021, pp. 874–880.

[4] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, "Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions," in *Proc. Annual Meeting of the ACL*, Toronto, 2023, pp. 10014–10037.

[5] O. Press, M. Zhang, S. Min, L. Schmidt, N. A. Smith, and M. Lewis, "Measuring and narrowing the compositionality gap in language models," in *Findings of EMNLP*, 2023.

[6] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1–38, 2023.

[7] A. Kamath, R. Jia, and P. Liang, "Selective question answering under domain shift," in *Proc. Annual Meeting of the ACL*, 2020, pp. 5684–5696.

[8] S. Longpre, K. Perisetla, A. Chen, N. Ramesh, C. DuBois, and S. Singh, "Entity-based knowledge conflicts in question answering," in *Proc. EMNLP*, 2021, pp. 7052–7063.

[9] X. Ho, A.-K. D. Nguyen, S. Sugawara, and A. Aizawa, "Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps," in *Proc. COLING*, 2020, pp. 6609–6625.

[10] M. Geva, D. Khashabi, E. Segal, T. Khot, D. Roth, and J. Berant, "Did Aristotle use a laptop? A question answering benchmark with implicit reasoning strategies," *Transactions of the ACL*, vol. 9, pp. 346–361, 2021.

[11] I. Stelmakh, Y. Luan, B. Dhingra, and M.-W. Chang, "ASQA: Factoid questions meet long-form answers," in *Proc. EMNLP*, 2022, pp. 8273–8288.

[12] H. Hayashi, P. Budania, P. Wang, C. Ackerson, R. Neervannan, and G. Neubig, "WikiAsp: A dataset for multi-domain aspect-based summarization," *Transactions of the ACL*, vol. 9, pp. 211–225, 2021.

[13] J. Lin, X. Ma, S.-C. Lin, J. Yang, R. Pradeep, and R. Nogueira, "Pyserini: A Python toolkit for reproducible information retrieval research with sparse and dense representations," in *Proc. ACM SIGIR*, 2021, pp. 2356–2362.

[14] Qwen Team, Alibaba Cloud, "Qwen3 Technical Report," *arXiv preprint arXiv:2505.09388*, 2025. [Online]. Available: https://arxiv.org/abs/2505.09388

[15] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2024.